

Proyecto de Forecasting de Ventas

Resumen de la fase de entrenamiento (entrenamiento.ipynb)

Preparado para: Diego Ramírez Cruz

Curso de iniciación a Machine Learning aplicado — revisión y mejora del
proyecto

Este documento recoge, de forma explicada, todas las decisiones tomadas en la fase de entrenamiento del modelo: tanto lo desarrollado originalmente en el curso como las mejoras, correcciones de errores y ajustes de rigor aplicados durante la revisión conjunta.

0. Contexto y objetivo del proyecto

El objetivo del proyecto es predecir las ventas futuras (**unidades_vendidas**) de un catálogo de productos de retail a partir de datos históricos de ventas (2021-2024) y de precios de la competencia (Amazon, Decathlon, Deporvillage). El resultado de esta fase será un modelo de Machine Learning entrenado, evaluado y validado con rigor, listo para generar predicciones sobre datos nuevos (noviembre 2025) en la fase de inferencia (*forecasting.ipynb*).

Se ha optado por un **único modelo global** (todos los productos a la vez, con la identidad del producto como variable más) en vez de un modelo independiente por producto o por tipo de día. Con un dataset de este tamaño (unos pocos miles de filas), un modelo global permite compartir el aprendizaje entre productos y periodos, mientras que modelos segmentados se quedarían con muy pocos datos cada uno y tenderían a sobreajustar.

1. Preparación y calidad de datos (fase previa al chat)

Antes de esta revisión ya se habían completado los siguientes pasos: carga de los datos de ventas y de precios de competencia desde CSV, un informe de calidad de datos (valores nulos, tipos, rangos), conversión de la columna **fecha** a formato datetime, y la unión (*merge*) de ambos datasets por fecha y producto mediante un **inner join**.

También se realizó un análisis exploratorio (EDA) completo: evolución temporal de ventas por año con foco en el pico de Black Friday, ventas por día de la semana, por categoría, por subcategoría, top 15 productos más vendidos, y comparación de la densidad de precios propios frente a los de la competencia.

■ **Concepto:** Un **inner join** conserva solo las filas cuya clave (fecha + producto) existe en ambos datasets. Es la opción correcta aquí: si un día no hay precio de competencia registrado para un producto, no tiene sentido mantener esa fila a medias, ya que faltaría información necesaria para las variables de precio relativo que se calculan más adelante.

2. Variables temporales y de calendario (fase previa al chat)

A partir de la columna **fecha** se generaron variables de calendario: **año**, **mes**, **día_mes**, **día_semana** (texto), **día_semana_num** (0-6), **trimestre**, **día_año** y **días_hasta_fin_año**. Además, banderas booleanas: **es_fin_semana**, **es_lunes**, **es_viernes**, **es_sabado**, **es_domingo**, **es_festivo**, **es_black_friday** y **es_cyber_monday**.

■ **Concepto:** Los modelos de árboles (como el que se usa aquí) no entienden fechas de forma nativa: no saben que el 25 de diciembre es Navidad ni que noviembre suele tener más ventas por el Black Friday. Descomponer la fecha en variables numéricas y booleanas explícitas es lo que le da al modelo la capacidad de aprender esos patrones estacionales mediante splits sobre esas columnas.

3. Variables de descuento y precio de competencia

Revisado y corregido durante este chat

3.1 descuento_porcentaje

Variable que mide la diferencia porcentual entre el precio base y el precio de venta real.

```
df['descuento_porcentaje'] = (
    (df['precio_base'] - df['precio_venta']) / df['precio_base']
```

```
) * 100
```

Problema detectado y corregido

La fórmula original tenía el signo invertido: **(precio_venta - precio_base) / precio_base**. Con esa versión, un descuento real (precio_venta < precio_base) daba un valor **negativo**, lo cual es contraintuitivo para una variable llamada 'descuento'. Además, esto provocó que el análisis automático de correlación del propio notebook etiquetara como 'inesperada' una correlación que en realidad era totalmente lógica (a mayor descuento real, más ventas). Se invirtió el signo para que un valor positivo signifique 'hay descuento real', coherente con el nombre de la variable.

También se añadió una comprobación de seguridad antes del cálculo, para detectar si algún **precio_base** fuera 0 y evitar una división por cero silenciosa (`n_ceros = df['precio_base'].eq(0).sum()`). Resultado tras el fix: media 1.13%, mediana 0.43%, rango de -3% a 15% de descuento.

3.2 precio_competencia y ratio_precio

```
df['precio_competencia'] = df[['Amazon', 'Decathlon', 'Deporvillage']].mean(axis=1)
df['ratio_precio'] = df['precio_venta'] / df['precio_competencia']
```

ratio_precio normaliza el precio propio frente al precio medio del mercado: valores por encima de 1 indican que se vende más caro que la competencia, y por debajo de 1, más barato. Es una variable más informativa para el modelo que el precio absoluto, ya que generaliza mejor entre productos de gamas de precio muy distintas.

Decisión de rigor aplicada: multicolinealidad

Tras calcular **descuento_porcentaje**, **precio_competencia** y **ratio_precio**, se eliminaron del dataframe las columnas **Amazon**, **Decathlon**, **Deporvillage**, **precio_venta** y **precio_competencia**. Se mantienen únicamente **precio_base**, **descuento_porcentaje** y **ratio_precio** como variables de precio.

■ **Concepto:** La **multicolinealidad determinística** ocurre cuando una variable es una combinación exacta de otras que también están en el dataset (por ejemplo, precio_venta, precio_base y descuento_porcentaje: con dos de ellas se deduce la tercera). En un modelo de árboles esto no rompe el entrenamiento, pero sí diluye la importancia entre columnas redundantes y complica la interpretación posterior (por ejemplo, el análisis de importancia de variables). Por eso se eliminan las columnas 'origen' una vez calculadas las variables derivadas, quedándose solo con la combinación mínima de variables no redundantes.

4. Encoding de variables categóricas

Revisado y corregido durante este chat

Se crean copias con sufijo **_h** de las variables categóricas relevantes, para poder aplicar one-hot encoding sobre las copias sin perder las columnas originales legibles (útiles para análisis y gráficos posteriores).

```
df['nombre_h'] = df['nombre']
df['categoria_h'] = df['categoria']
df['subcategoria_h'] = df['subcategoria']

df = pd.get_dummies(
    df, columns=['nombre_h', 'categoria_h', 'subcategoria_h'],
    drop_first=False
)
```

Dos variables descartadas del encoding

- **día_semana_h**: se detectó que era información totalmente redundante — el día de la semana ya está capturado por **día_semana_num** (ordinal) y por las banderas **es_lunes/es_viernes/es_sabado/es_domingo/es_fin_semana**. Codificarlo una tercera vez solo añadía ruido y colinealidad.
- **nombre_festivo_h**: la columna original solo tenía 96 valores no nulos de 3.384 filas (festivos concretos, cada uno con apenas un puñado de apariciones en 4 años). Un one-hot sobre eso genera muchas columnas casi vacías, con muy poca capacidad de aprendizaje individual, y esa señal ya está mejor representada por **es_festivo**, **es_black_friday** y **es_cyber_monday**.

■ **Concepto**: El **one-hot encoding** convierte una variable categórica de texto en varias columnas binarias (0/1), una por cada categoría posible, porque los modelos de Machine Learning necesitan números, no texto. El parámetro **drop_first=False** mantiene todas las categorías: en una regresión lineal esto causaría un problema de colinealidad perfecta (la 'trampa de la variable dummy'), pero en modelos de árboles como el usado aquí no es un problema, y conservarlas todas facilita la interpretación posterior por categoría.

5. Variables de lags y media móvil — fuga de información detectada y corregida

Hallazgo crítico de este chat

Esta fue la corrección más importante de toda la revisión: el código original generaba variables de lag (ventas de días anteriores) y media móvil con dos errores que provocaban **fuga de información (data leakage)**, inflando artificialmente el rendimiento del modelo.

5.1 Código original (con los dos bugs)

```
df = df.sort_values(['año', 'fecha', 'producto_id']).reset_index(drop=True)

for año in sorted(df['año'].unique()):
    mask_año = df['año'] == año
    for lag in range(1, 8):
        df.loc[mask_año, f'lag_{lag}_unidades_vendidas'] = \
            df.loc[mask_año, 'unidades_vendidas'].shift(lag)
    df.loc[mask_año, 'media_movil_7_unidades_vendidas'] = \
        df.loc[mask_año, 'unidades_vendidas']\
            .rolling(window=7, min_periods=1).mean()
```

- **Bug 1 — mezcla de productos**: el orden usado (año, fecha, producto_id) coloca todos los productos de un mismo día seguidos. Un **shift(1)** sobre esos datos no coge 'lo que vendió este producto ayer', sino 'lo que vendió el producto anterior en la lista, ese mismo día'. Los supuestos lags no eran lags temporales reales del mismo producto.
- **Bug 2 — la media móvil se mira a sí misma**: faltaba un **shift(1)** antes del **rolling()**. Sin él, la ventana de 7 días incluye el valor de la propia fila que se está prediciendo. En la primera fila de cada segmento (con **min_periods=1**), la media móvil era literalmente idéntica al valor real de ventas de ese día — el predictor contenía la respuesta.

■ **Concepto**: El **data leakage** (fuga de información) ocurre cuando una variable predictora contiene, directa o indirectamente, información que en un escenario real de predicción no estaría disponible en el momento de predecir — en este caso, el propio valor que se quiere predecir, o el de otro producto del mismo día que tampoco se conocería de antemano. Es uno de los errores más peligrosos en forecasting porque no rompe el código ni da errores: al contrario, hace que las métricas de validación parezcan mucho mejores de lo que el modelo realmente sería en producción. La señal de alarma que lo delató aquí fue un R^2 sospechosamente alto (0,93) para un problema de demanda retail.

5.2 Código corregido

```
df = df.sort_values(['producto_id', 'fecha']).reset_index(drop=True)

for lag in range(1, 8):
    df[f'lag_{lag}_unidades_vendidas'] = (
        df.groupby('producto_id')['unidades_vendidas'].shift(lag)
    )

df['media_movil_7_unidades_vendidas'] = (
    df.groupby('producto_id')['unidades_vendidas']
        .transform(lambda s: s.shift(1).rolling(window=7, min_periods=1).mean())
)

df = df.dropna(subset=[f'lag_{i}_unidades_vendidas' for i in range(1, 8)]
               + ['media_movil_7_unidades_vendidas'])
```

El **groupby('producto_id')** garantiza que cada shift y cada media móvil se calculan dentro de la serie temporal de un único producto. Como efecto secundario positivo, al no reiniciar los lags en cada cambio de año (el código original sí lo hacía), se recuperan registros de 2022-2024 que antes se perdían de forma innecesaria.

Impacto real del fix — comparación de las métricas del modelo final antes y después de corregir el leakage:

Métrica	Con leakage (antes)	Sin leakage (después, real)
MAE	0.759	1.002
RMSE	1.659	1.935
MAPE	16.60%	20.97%
R ²	0.9296	0.9040

El empeoramiento es real pero moderado, no catastrófico — señal de que el modelo seguía aprendiendo patrones genuinos (precio, estacionalidad, tendencia por producto) además de la fuga, y no dependía enteramente de ella.

6. Entrenamiento del modelo

6.1 División train / validation

```
train_df = df[df['año'].isin([2021, 2022, 2023]).copy()]
validation_df = df[df['año'] == 2024].copy()
```

■ **Concepto:** En series temporales **no se puede dividir aleatoriamente** entre entrenamiento y validación: si se mezclan fechas de 2024 en el entrenamiento y de 2021 en la validación, el modelo estaría aprendiendo con información del futuro respecto a lo que se le pide predecir. Por eso se divide por corte cronológico estricto: todo 2024 queda después de todo el periodo de entrenamiento, tal y como ocurrirá en producción.

6.2 Selección de variables predictoras — bug corregido

El filtro automático original (*dtype != 'object'*) no excluía las columnas de texto en el entorno de pandas usado, porque su dtype real es **'str'**, no **'object'**. Esto provocó un error de entrenamiento al intentar usar **producto_id** como número. Se sustituyó por una lista explícita de exclusión, más robusta y auditable:

```
cols_excluir = ['fecha', 'ingresos', 'unidades_vendidas',
                'producto_id', 'nombre', 'categoria', 'subcategoria',
                'día_semana', 'nombre_festivo', 'año']

predictor_vars = [col for col in train_df.columns if col not in cols_excluir]
```

ingresos se excluye porque se calcula a partir de la propia variable objetivo (*unidades_vendidas × precio*), así que incluirla sería fuga de información directa. **año** se excluyó tras el análisis de importancia de variables: un árbol de decisión no puede extrapolar más allá del rango de valores vistos en entrenamiento (2021-2023), así que en noviembre 2025 trataría 'año' como si fuera 2023, perdiendo cualquier tendencia real entre años.

6.3 El modelo: HistGradientBoostingRegressor

■ **Concepto:** Es un modelo de **gradient boosting**: construye muchos árboles de decisión pequeños de forma secuencial, donde cada árbol nuevo se centra en corregir los errores que cometieron los árboles anteriores. Es una de las familias de modelos más efectivas para datos tabulares de tamaño pequeño-mediano como este: captura relaciones no lineales e interacciones entre variables de forma automática, no requiere escalar los datos (a diferencia de una regresión lineal), y tolera razonablemente bien la colinealidad entre variables.

6.4 Walk-forward validation con TimeSeriesSplit

En vez de evaluar el modelo con un único corte fijo, se usa **TimeSeriesSplit(n_splits=5)** sobre **train_df** (2021-2023), que genera varias particiones respetando siempre el orden cronológico: en cada partición, el entrenamiento crece y la validación son siempre las filas cronológicamente posteriores.

■ **Concepto:** La **validación walk-forward** (o de ventana expansiva) da una estimación mucho más fiable del rendimiento real que un único split, porque promedia el error a lo largo de varios periodos en vez de depender de que el periodo de validación elegido haya sido, por azar, 'fácil' o 'difícil'. Es el estándar recomendado en forecasting profesional frente a una sola partición fija.

6.5 Búsqueda automática de hiperparámetros: RandomizedSearchCV

```
param_dist = {
    'learning_rate': uniform(0.01, 0.19),
    'max_iter': randint(150, 600),
    'max_leaf_nodes': randint(15, 63),
    'max_depth': randint(3, 12),
    'min_samples_leaf': randint(10, 50),
    'l2_regularization': uniform(0, 3),
```

```
}

search = RandomizedSearchCV(
    estimator=base_model, param_distributions=param_dist,
    n_iter=40, scoring='neg_mean_absolute_error',
    cv=TimeSeriesSplit(n_splits=5), random_state=42, n_jobs=-1
)
search.fit(X_train, y_train)
```

■ **Concepto:** En vez de fijar los hiperparámetros a mano por prueba y error, **RandomizedSearchCV** prueba automáticamente muchas combinaciones aleatorias dentro de los rangos definidos y se queda con la que mejor puntúa. El detalle importante aquí es que se usa el mismo esquema walk-forward (**TimeSeriesSplit**) también como validación interna de la búsqueda — si se usara una validación cruzada aleatoria normal, se estaría filtrando información temporal también durante el ajuste de hiperparámetros, no solo en la evaluación final. El año 2024 (validation_df) no participa en ningún momento de esta búsqueda: se reserva exclusivamente para la evaluación final, una sola vez.

Mejores hiperparámetros encontrados:

Hiperparámetro	Valor
learning_rate	0.191
max_iter	338
max_depth	10
max_leaf_nodes	35
min_samples_leaf	48
l2_regularization	1.124
MAE promedio en CV (walk-forward)	1.463 unidades

7. Resultados finales y comparación con baselines

■ **Concepto:** Un resultado de error (MAE = 1 unidad, por ejemplo) no significa nada por sí solo: hay que compararlo contra un **baseline** — una estrategia de predicción trivial — para saber si el modelo realmente aporta valor. Se usaron dos: el **naive de la media** (predecir siempre la media histórica de ventas) y el **naive estacional** (predecir 'lo mismo que se vendió hace 7 días', usando la propia variable `lag_7`), este último mucho más exigente.

Métrica	Modelo final	Naive estacional (<code>lag_7</code>)	Naive media
MAE	1.002	3.011	3.420
RMSE	1.935	7.458	—
MAPE	20.97%	—	—
R ²	0.9040	-0.4250	—

El modelo reduce el error frente al baseline más exigente (naive estacional) en aproximadamente un **67%** (MAE de 3.01 a 1.00), y pasa de un R² negativo (el naive estacional ni siquiera supera a predecir la media constante) a un R² de 0.904. Es una mejora clara e inequívoca.

Qué mide cada métrica

- **MAE (Mean Absolute Error):** error medio en las mismas unidades que la variable objetivo (unidades vendidas) — el más fácil de comunicar al negocio.
- **RMSE (Root Mean Squared Error):** como el MAE, pero penaliza más los errores grandes. Es mayor que el MAE porque hay algunos errores puntuales más grandes (probablemente en picos de demanda como Black Friday, difíciles de acertar al 100%).
- **MAPE (Mean Absolute Percentage Error):** error medio en porcentaje. Es la métrica que mejor entiende un negocio ('nos equivocamos un 21% de media'), pero es más sensible en productos con ventas bajas, donde un pequeño error absoluto se traduce en un gran error relativo.
- **R² (coeficiente de determinación):** proporción de la variabilidad de las ventas que el modelo logra explicar. Un valor de 0.90 significa que el modelo captura el 90% de esa variabilidad.

8. Importancia de variables (permutation importance)

■ **Concepto:** La **importancia por permutación** mide cuánto empeora el error del modelo cuando se baraja aleatoriamente (se destruye la relación con el objetivo) una columna a la vez, evaluado siempre sobre datos que el modelo no ha visto en entrenamiento (`validation_df`, 2024). Es más fiable que la importancia nativa de los árboles (`feature_importances_`), que está sesgada a favor de variables con muchos valores distintos y mide utilidad durante el propio entrenamiento, no capacidad real de generalizar.

```
result = permutation_importance(
    modelo_final, X_val, y_val,
    n_repeats=20, random_state=42,
    scoring='neg_mean_absolute_error', n_jobs=-1
)
```

Las columnas del one-hot encoding (`nombre_h_*`, `categoria_h_*`, `subcategoria_h_*`) se agruparon por variable original, ya que su importancia individual queda repartida entre decenas de columnas.

Variable / grupo	Importancia media
lag_1_unidades_vendidas	0.870
descuento_porcentaje	0.598
lag_7_unidades_vendidas	0.554
ratio_precio	0.223
día_año	0.162
es_black_friday	0.156
media_movil_7_unidades_vendidas	0.148
mes	0.077
nombre_h (agrupado)	0.054
categoría_h (agrupado)	0.042

Lectura de negocio

- La **inercia reciente** domina claramente (lag_1 y lag_7): lo que se vendió ayer y hace una semana es, con diferencia, la mejor señal de lo que se venderá hoy.
- El **precio propio** (descuento_porcentaje) y el **precio relativo a la competencia** (ratio_precio) son drivers reales confirmados con un método riguroso, no solo por correlación simple.
- La **estacionalidad** (día_año, mes) y los **eventos comerciales** (es_black_friday) también son relevantes.
- La **identidad del producto o categoría** (nombre_h, categoría_h) aporta relativamente poco una vez que el modelo ya conoce los lags de ese mismo producto — su efecto ya queda capturado indirectamente a través del historial reciente de ventas.

■ **Concepto:** Limitación importante del método: si una variable apenas varía dentro del conjunto de validación (por ejemplo, es_cyber_monday, con muy pocos días positivos al año), barajarla apenas cambia el error y su importancia sale artificialmente baja o nula, sin que eso signifique que el modelo no la use en absoluto.

9. Conclusiones y estado final del proyecto

La fase de entrenamiento queda cerrada con un modelo riguroso y defendible: sin fuga de información, con una selección de variables libre de colinealidad determinística innecesaria, validado con un esquema walk-forward y con hiperparámetros ajustados automáticamente. El modelo final reduce el error un ~67% frente al mejor baseline disponible y su comportamiento (según el análisis de importancia) es coherente con el sentido común del negocio: inercia de demanda, precio y estacionalidad como principales impulsores.

Principales mejoras aplicadas respecto al proyecto de curso original

- Corrección del signo de descuento_porcentaje para que sea interpretable.
- Eliminación de colinealidad determinística en variables de precio (precio_venta, precio_competencia).
- Eliminación de codificaciones redundantes en el encoding (día de la semana, festivos dispersos).

- Corrección de un bug crítico de data leakage en los lags y la media móvil.
- Sustitución de un filtro de selección de variables poco fiable por una exclusión explícita.
- Paso de un único split a validación walk-forward con TimeSeriesSplit.
- Búsqueda automática de hiperparámetros con RandomizedSearchCV, respetando el orden temporal.
- Eliminación de 'año' como predictora por su incapacidad de extrapolación en modelos de árboles.
- Incorporación de un baseline naive estacional, más exigente que el naive de la media.
- Análisis de importancia de variables por permutación para validar la coherencia del modelo.

Próximo paso: fase de inferencia sobre datos nuevos (forecasting.ipynb), aplicando exactamente el mismo pipeline de transformación de variables validado en este documento.